Visvesvaraya Technological University, Belagavi

Alva's Institute of Engineering and Technology

Moodbidri, Karnataka

Department of Computer Science and Engineering

A PROJECT REPORT ON

Early Detection and Management of Cow Diseases

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering in Computer Science and Engineering

Submitted by

Sinchana (4AL23CS156) **Manisha K(4AL23CS078)**

Under the guidance of

Mr.Anijith Raja R

Academic Year (2026-2027)

CERTIFICATE

This is to certify that the project work entitled "GoShala Care: Early Detection and Management of Cow Diseases" is a bonafide work carried out by Sinchana (4AL23CS156) and Manisha K (4AL23CS078), in partial fulfilment for the award of the degree of Bachelor of Engineering in Computer Science and Engineering of Visveswaraya Technological university,Belagavi during the academic year 2026-2027. It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the Bachelor of Engineering degree.

_____

Mr.Anjith Raja R

Project Guide

External Viva

Name of the Examiners	Signature with date
1. _____	_____
2. _____	_____

DECLARATION

We, Sinchana S(4AL23CS156) and Manisha K(4AL23CS078), a student of the final year Bachelor of Engineering in Computer Science and Engineering at Alva's Institute of Engineering and Technology, Moodbidri, hereby declare that the project work entitled "GoShala Care: Early Detection and Management of Cow Diseases" has been carried out by me under the guidance of Mr. Anjith Raja R, Department of Computer Science and Engineering, and is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering of Visweshvaraya Technological Unversity,Belagavi during the academic year 2026-2027.

I further declare that this report has not been submitted previously for the award of any degree, diploma or similar title to any other university or institution.

Place: Mijar
Date: 30/09/2026

Sinchana S
Manisha K

ACKNOWLEDGEMENT

We express my sincere gratitude to my project guide, Mr.Anjith Raja R Department of Computer Science and Engineering, for the valuable guidance, encouragement and support given throughout this project.

We thank Dr. Manjunath Kotari, Head of the Department of Computer Science and Engineering, and Dr.Peter Fernandes, Principal, Alva's Institute of Engineering and Technology, for providing the facilities and an encouraging environment for carrying out this work.

We are thankful to all the faculty and staff of the department, to the veterinarians and farmers who shared their knowledge and experience, and to my friends and family for their constant support and motivation.

Sinchana S

Manisha K

ABSTRACT

Diseases in dairy cattle cause significant losses to farmers through reduced milk production, treatment costs and the spread of infection, and the losses are greatest when disease is detected late. Existing research on early detection mainly relies on IoT wearables, infrared thermography and sensor-based machine learning, which require costly hardware and are rarely delivered as a product that farmers can use.

This project presents GoShala Care, a role-based web application built with Flask and MySQL for early detection and management of cow diseases without special hardware. Farmers record daily temperature, feed intake, water intake and milk yield, and a symptom checklist for each cow. The system combines rule-based thresholds with a trained machine learning model, a tabular classifier for vitals and symptoms and a CNN with transfer learning for skin and udder photographs, to produce a Low, Moderate or High risk score. Moderate and High risk cases are automatically queued for veterinarian review. The system has three roles: Farmer, Vet and Admin, where the Admin manages veterinarian accounts, a map-based veterinarian directory and a disease and symptom library.

The system was evaluated through functional testing of each role and machine learning metrics including precision, recall and F1-score. The project shows that an affordable, explainable and vet-in-the-loop system can support early disease detection on small and medium farms.

Keywords: cattle disease detection, early warning, Flask, MySQL, machine learning, CNN, transfer learning, role-based web application.

TABLE OF CONTENTS

Chapter 1: Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Objectives 2
1.5 Scope of the Project 2
1.6 Organization of the Report 3
Chapter 2: Literature Survey **4**
2.1 Overview 4
2.2 IoT Wearables and Collars 4
2.3 Infrared Thermography 4
2.4 Sensor-Based Machine Learning Classifiers 4
2.5 Image-Based Disease Detection 4
2.6 Summary of Reviewed Work 5
2.7 Research Gap 5
Chapter 3: System Analysis **6**
3.1 Existing System 6
3.2 Limitations of the Existing System 6
3.3 Proposed System 6
3.4 Comparison of Existing and Proposed Systems 6
3.5 Feasibility Study 7
3.6 Key Differentiators 7
Chapter 4: System Requirements **8**
4.1 Hardware Requirements 8
4.2 Software Requirements 8
4.3 Technology Stack 8
4.4 Functional Requirements 8
4.5 Non-Functional Requirements 9
Chapter 5: System Design **10**
5.1 Design Overview 10
5.2 System Architecture 10
5.3 Database Design 11
5.4 Use Case Diagram 11
5.5 Data Flow Diagrams 12
5.6 Sequence Diagram 13
Chapter 6: Implementation **14**
6.1 Module Overview 14
6.2 Authentication and Role-Based Access Control 14
6.3 Herd and Vitals Management 14
6.4 Symptom Checker and Risk Scoring 14
6.5 Photo Scan Module 15
6.6 Disease Library, Vet Directory and Admin Panel 16
6.7 Veterinarian Review Workflow 16
Chapter 7: Data Science Methodology **17**
7.1 Overview 17
7.2 Data Cleaning 17
7.3 Feature Engineering 17
7.4 Model Selection and Training 18
7.5 Evaluation Metrics 18
7.6 Interpretation of Results 18
Chapter 8: Testing **19**
8.1 Testing Strategy 19
8.2 Test Cases 19
8.3 Observations 19
Chapter 9: Results and Discussion **20**
9.1 Application Screens 20

9.2 Discussion....21
9.3 Limitations....22
Chapter 10: Conclusion and Future Scope....23
10.1 Conclusion....23
10.2 Future Scope....23
References....24
Appendix....25
A. Database Schema....25

LIST OF FIGURES

Figure 5.1: System architecture of GoShala Care(1)....10
Figure 5.2: System architecture (2)....10
Figure 5.3: Entity relationship diagram....11
Figure 5.4: Use case diagram for Farmer, Vet and Admin....11
Figure 5.5: Data flow diagram, level 0 (context diagram)....12
Figure 5.6: Data flow diagram, level 1....12
Figure 5.7: Sequence diagram: symptom check, risk score and vet review....13
Figure 6.1: Flowchart of the hybrid risk scoring process....14
Figure 7.1: Machine learning pipeline for tabular and image models....17
Figure 9.1: Farmer DashBoard page....20
Figure 9.2: LogDaily Vitals Page....20
Figure 9.3: Photo Scan page....21
Figure 9.4: Vaccination Details page....21

LIST OF TABLES

Table 2.1: Summary of reviewed literature....5
Table 3.1: Comparison of existing and proposed systems....6
Table 4.1: Hardware requirements....8
Table 4.2: Software requirements....8
Table 4.3: Functional requirements....8
Table 6.1: Modules of GoShala Care....14
Table 8.1: Test cases....19

CHAPTER 1: INTRODUCTION

1.1 Background

Dairy farming is a major source of livelihood for millions of rural households in India, and the health of the herd directly determines the income of the farmer. A sick cow produces less milk, requires costly treatment, and in the case of contagious diseases can infect the rest of the herd. Conditions such as mastitis, lumpy skin disease, foot-and-mouth disease, fever-causing infections and metabolic disorders are common, and their impact is greatest when they are noticed late.

Most small and medium farmers rely on visual observation and experience to judge whether an animal is unwell. Subtle early signs such as a small rise in body temperature, a drop in feed or water intake, or a gradual fall in milk yield are easy to miss, especially on farms with many animals. By the time obvious symptoms appear, the disease has often progressed, treatment is more expensive, and the animal may already have spread infection. Veterinary doctors are also few in number in rural regions, so they usually see a case only after it has become serious.

Research on cattle health monitoring has therefore explored sensor-based systems that track temperature, activity and rumination continuously. Although technically promising, these systems depend on specialised hardware that many farmers cannot afford or maintain. GoShala Care is designed to bring early detection and case management to farmers using only a web browser and the data that farmers already observe every day.

1.2 Problem Statement

Cattle diseases can significantly affect animal health, milk production, productivity and the economic condition of farmers if they are not identified at an early stage. In many farms, cattle health is primarily monitored through manual observation, making it difficult to continuously track important parameters such as temperature, feed intake, water intake, milk production and observed symptoms. Subtle changes in a cow's normal health condition may go unnoticed until the condition becomes more serious, while timely veterinary assistance may not always be available. Existing automated cattle-health monitoring systems often depend on IoT sensors, wearable devices, thermal cameras and other specialised hardware, which can increase implementation cost and maintenance requirements. In addition, health records may not be maintained systematically, making it difficult to analyse changes in an individual cow's health over time. Therefore, there is a need for an accessible software-based solution that can combine multiple health parameters and symptoms with machine-learning and rule-based analysis to identify potential health risks at an early stage. The system should provide understandable risk information, generate alerts for cases requiring attention, and enable veterinarians to review higher-risk cases and provide appropriate guidance

1.3 Motivation

The motivation behind **GoShala Care** is to develop a practical and accessible solution for improving cattle-health monitoring and supporting the early identification of potential diseases. Farmers often depend on manual observation and periodic veterinary assistance, which may make it difficult to identify subtle changes in a cow's health at the right time. Existing smart monitoring solutions can require specialised sensors and hardware, making them more complex and costly to adopt. GoShala Care aims to address these challenges through a software-based platform that records important health information, analyses symptoms and health parameters using rule-based and machine-learning techniques, and highlights cattle that may require attention. By connecting farmers with veterinarians through a common platform, the project aims to support timely intervention, organised health-record management and better overall cattle-health management.

1.4 Objectives

The main objectives of the project are:

- To develop a role-based web application with separate interfaces for Farmer, Veterinarian and Admin users.
- To allow farmers to record daily vitals (temperature, feed intake, water intake, milk yield) and a symptom checklist for each cow.
- To build a hybrid risk engine that combines rule-based thresholds with a trained machine learning classifier to produce a Low, Moderate or High risk score.
- To support photo-based screening of skin and udder conditions using a CNN built with transfer learning.
- To automatically queue Moderate and High risk cases for veterinarian review and return the veterinarian's advice to the farmer.
- To provide a disease and symptom reference library and a map-based veterinarian directory managed by the Admin.

1.5 Scope of the Project

The scope of **GoShala Care** includes cattle registration, health-record management, daily health monitoring, symptom checking, risk assessment and photo-based screening of selected visible conditions. The system allows farmers to record important health parameters and receive Low, Moderate or High-risk assessments using rule-based and machine-learning techniques. Moderate- and High-risk cases can be forwarded to veterinarians for review and recommendations. The system also provides administrative functions for managing users, veterinarians, disease information and reference data. GoShala Care is designed as a **screening and decision-support system** and does not replace professional veterinary diagnosis. The current system does not depend on wearable sensors or specialised IoT hardware, while IoT-based automatic monitoring can be considered as future enhancement.

1.6 Organization of the Report

- **Chapter 1 – Introduction:** Presents the background, problem statement, motivation, objectives and scope of GoShala Care.
- **Chapter 2 – Literature Survey:** Reviews existing research on cattle-health monitoring, IoT, machine learning, infrared thermography and image-based disease detection.
- **Chapter 3 – System Analysis:** Discusses the existing system, proposed system and identified research gap.
- **Chapter 4 – System Requirements:** Describes the hardware, software, functional and non-functional requirements.
- **Chapter 5 – System Design:** Presents the system architecture, database design, use-case diagram, data-flow diagrams and sequence diagrams.
- **Chapter 6 – Implementation:** Explains the implementation of the major modules and application functionality.
- **Chapter 7 – Data Science Methodology:** Describes the datasets, preprocessing, feature engineering, model training and evaluation.
- **Chapter 8 – Testing:** Presents the test cases, testing process and observations.
- **Chapter 9 – Results and Discussion:** Discusses the system outputs, major screens, results and limitations.
- **Chapter 10 – Conclusion and Future Scope:** Summarises the achievements of the project and presents possible future enhancements.
- **References:** Lists the research papers, datasets and other sources used in the project.
- **Appendix:** Provides additional technical information and supporting material.

CHAPTER 2: LITERATURE SURVEY

2.1 Overview

Automated monitoring of cattle health has been studied extensively under precision livestock farming. Existing research includes wearable wireless biosensors, infrared thermography, machine-learning-based disease prediction and image-based deep-learning approaches. A systematic review of machine learning in dairy-farm management found that 55% of the reviewed primary studies focused on disease detection, demonstrating the significant role of machine learning in automated cattle-health assessment [4]. Reviews of cattle wearable systems and infrared thermography similarly show increasing use of sensing and artificial intelligence for continuous or non-invasive health monitoring [1]–[3]. More recent work has also explored deep learning for image-based cattle disease detection [5], [6].The following sections examine these approaches, their reported advantages and limitations, and their relevance to the development of GoShala Care.

2.2 IoT Wearables and Collars

Wearable devices such as collars, ear tags and leg bands use sensors to monitor cattle activity, temperature, feeding and rumination. These systems enable continuous health monitoring, but they require additional hardware, wireless communication and device maintenance [1], [2]. GoShala Care reduces this hardware dependency by allowing farmers to enter health information through a web application.

2.3 Infrared Thermography

Infrared thermography provides non-contact measurement of cattle surface temperature and has been investigated for conditions such as mastitis, lameness and fever. Research has shown that thermal images can be combined with machine learning for cattle-health assessment [3], [7]. However, thermal cameras and suitable imaging conditions are required, so this technology is considered a possible future extension of GoShala Care.

2.4 Sensor-Based Machine Learning Classifiers

Machine learning has been widely applied to cattle-health and dairy-farm data. A systematic review found that disease detection was one of the major applications of machine learning in dairy farming, with methods such as decision trees and neural networks commonly investigated [4]. GoShala Care uses health parameters and symptoms as inputs to support machine-learning-based risk assessment.

2.5 Image-Based Disease Detection

Deep-learning techniques, particularly CNNs and transfer learning, have been applied to identify visible cattle diseases from images. Studies on lumpy skin disease demonstrate the potential of pretrained models for automated image-based detection [5], [6]. However, variations in image quality, lighting and dataset size can affect performance. GoShala Care therefore treats image analysis as a supporting component rather than a replacement for veterinary diagnosis.

2.6 Summary of Reviewed Work

Table 2.1: Summary of reviewed literature

S.No	Reference	Approach	Limitation
1	Park et al. [1]	Wearable wireless biosensors, collars, ear tags and other devices	Provides continuous physiological and behavioural monitoring; performance varies across parameters and experimental conditions.
2	Alipio and Villena [2]	Intelligent wearable devices and biosensors	Identifies benefits for real-time cattle-health monitoring but reports device complexity, wireless transmission difficulties and environmental influences.
3	Wang et al. [3]	Infrared thermography + ML/DL	Supports non-contact cattle-health assessment; thermal-image acquisition and environmental/methodological factors remain important considerations.
4	Slob et al. [4]	Machine learning for dairy-farm management	Disease detection is a major ML application; feature selection, imbalance, overfitting and parameter tuning are reported challenges.
5	Study on LSD detection [5]	CNN and transfer learning	Demonstrates the potential of pretrained deep-learning models for cattle image-based disease detection; dataset quality and generalisation remain important.

2.7 Research Gap

The reviewed work shows that early disease detection is feasible, but three gaps remain. First, most solutions depend on dedicated hardware that raises cost. Second, few solutions are delivered as a complete role-based product that includes farmers, veterinarians and administrators. Third, machine learning outputs are rarely combined with an explainable rule layer, which makes them harder for farmers to trust. GoShala Care addresses these gaps.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Existing System

In practice, most farmers monitor their cattle by observation and record keeping in notebooks, and contact a veterinarian only when a problem becomes obvious. In research and larger commercial farms, sensor-based systems using collars, ear tags or thermal cameras are used. Both situations have significant drawbacks.

3.2 Limitations of the Existing System

- Manual observation is subjective and misses early, subtle changes.
- Paper records are difficult to analyse for trends across days or across animals.
- Sensor-based systems require costly hardware, power, connectivity and maintenance.
- Research prototypes often lack a user interface for farmers and veterinarians.
- There is no structured way to route risky cases to a veterinarian for timely review.

3.3 Proposed System

GoShala Care is a role-based web application built with Flask and MySQL. Farmers log daily vitals and symptoms for each cow. The system applies rule-based thresholds and a trained machine learning model to compute a risk level of Low, Moderate or High. Photographs of skin or udder conditions can be analysed by a CNN. Moderate and High risk cases are automatically queued for veterinarian review, and the Admin manages veterinarian accounts, a map-based veterinarian directory and the disease and symptom reference library.

3.4 Comparison of Existing and Proposed Systems

Table 3.1: Comparison of existing and proposed systems

Aspect	Existing systems	GoShala Care
Hardware	Collars, ear tags, thermal cameras	None; browser only
Cost	High per-animal cost	Low; software only
Data input	Automatic sensor streams	Farmer-entered vitals, symptoms, photos
Analysis	Threshold or black-box ML	Rule layer plus ML layer (explainable)
Veterinarian role	Usually external to the system	Built-in review queue (vet-in-the-loop)
User roles	Often single-user prototypes	Farmer, Vet and Admin
Knowledge base	Rarely included	Disease library and vet directory with map

3.5 Feasibility Study

Technical feasibility: the system uses open-source tools (Python, Flask, MySQL, scikit-learn and TensorFlow/Keras) that run on ordinary computers. Economic feasibility: no hardware purchase is needed, so the development and running cost is low. Operational feasibility: the interface uses simple forms and checklists suitable for farmers with limited technical knowledge, and the veterinarian workflow is limited to reviewing queued cases.

3.6 Key Differentiators

- No special hardware: the system works with data the farmer already observes each day, so there is no per-animal device cost.
- A complete role-based web product: Farmer, Vet and Admin roles each have their own interface and permissions, rather than a single research prototype.
- Vet-in-the-loop: every Moderate or High risk case is placed in a veterinarian's review queue and the advice is returned to the farmer.
- Explainable rule layer plus ML layer: the rule engine gives reasons the farmer can understand, while the machine learning model adds learned patterns.

CHAPTER 4: SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Table 4.1: Hardware requirements

Component	Specification
Processor	Intel Core i5 or equivalent
RAM	8 GB or above
Storage	256 GB or above
Client device	Any computer or smartphone with a web browser
Network	Internet or local network access to the server

4.2 Software Requirements

Table 4.2: Software requirements

Component	Specification
Operating system	Windows / Linux
Programming language	Python 3.x
Web framework	Flask
Database	MySQL
ML libraries	scikit-learn, pandas, NumPy, TensorFlow/Keras, joblib
Frontend	HTML, CSS, JavaScript
Browser	Chrome, Edge or Firefox (latest versions)

4.3 Technology Stack

The backend is implemented in Python with the Flask micro-framework, which handles routing, sessions and templating. MySQL stores users, cows, logs, diseases and reports. The machine learning layer uses scikit-learn for the tabular classifier and TensorFlow/Keras for the image model, with trained models saved to disk and loaded by the Flask application at start-up. The frontend uses server-rendered templates with HTML, CSS and JavaScript.

4.4 Functional Requirements

Table 4.3: Functional requirements

ID	Requirement
FR1	The system shall allow users to register and log in and shall assign one of three roles: Farmer, Vet or Admin.
FR2	The system shall restrict each page and action to the roles authorised to use it.
FR3	A farmer shall be able to add, edit and view cows.

ID	Requirement
FR4	A farmer shall be able to log daily temperature, feed intake, water intake and milk yield for each cow.
FR5	A farmer shall be able to submit a symptom checklist and receive a Low, Moderate or High risk score.
FR6	A farmer shall be able to upload a photo for skin or udder screening.
FR7	The system shall automatically queue Moderate and High risk cases for veterinarian review.
FR8	A veterinarian shall be able to review queued cases and record a diagnosis and advice.
FR9	An admin shall be able to create and manage veterinarian accounts and the vet directory shown on a map.
FR10	An admin shall be able to manage the disease and symptom reference library.

4.5 Non-Functional Requirements

- Usability: simple forms and checklists that can be used with minimal training.
- Security: passwords stored as hashes, session-based authentication and role-based access control.
- Reliability: input validation and consistent storage of all logs in the database.
- Performance: risk scores returned within a few seconds of submission. Maintainability: modular code with separate modules for authentication, herd management, prediction and administration.
- Portability: accessible from any modern browser on desktop or mobile.

CHAPTER 5: SYSTEM DESIGN

5.1 Design Overview

The design follows a layered architecture that separates the presentation, application logic, intelligence and data layers. This chapter presents the architecture, the database design, the use cases, the data flow and the main interaction sequence.

5.2 System Architecture

The browser-based interfaces for the three roles communicate with the Flask backend. Authentication and role checking are applied to every request. The application modules pass data to the intelligence layer, where the rule engine, tabular classifier and CNN produce a risk score, and all results are stored in MySQL.

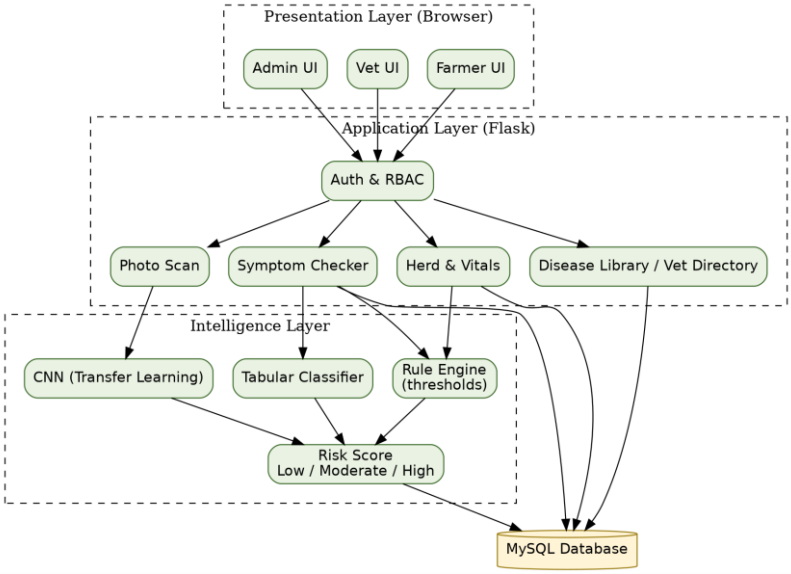


Figure 5.1: System architecture of GoShala Care(1)

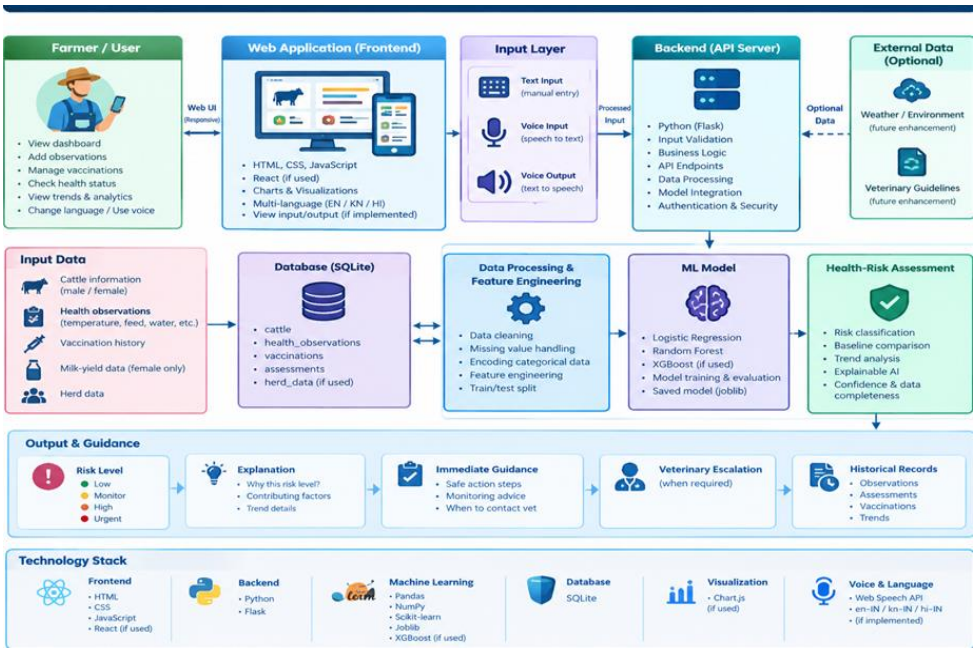


Figure 5.2: System Architecture(2)

5.3 Database Design

The database is relational. A user owns many cows; each cow has many vitals and symptom logs and many milk logs. Logs with Moderate or High risk generate reports assigned to veterinarians. The disease library links diseases and symptoms through the disease_symptoms table, and the vet directory stores clinic details and map coordinates. The complete schema is given in the Appendix.

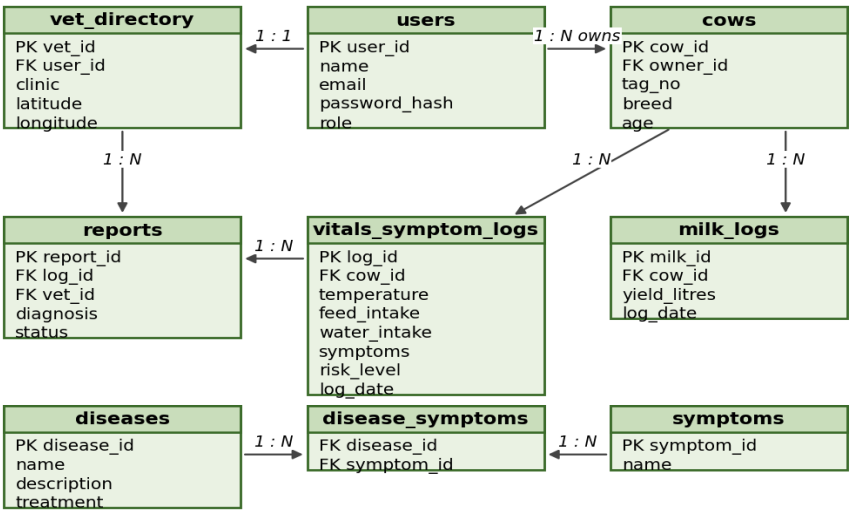


Figure 5.3: Entity relationship diagram

5.4 Use Case Diagram

The three actors are the Farmer, the Veterinarian and the Admin. All actors log in through the authentication module, after which their permitted functions are shown.

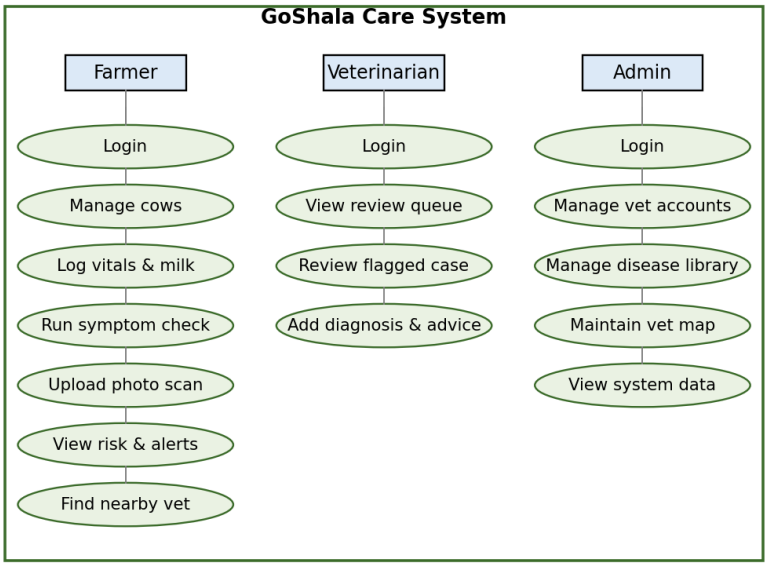


Figure 5.4: Use case diagram for Farmer, Vet and Admin

5.5 Data Flow Diagrams

The level 0 diagram shows the system as a single process interacting with three external entities. The level 1 diagram decomposes it into authentication, herd and log management, risk assessment, veterinarian review and admin management.

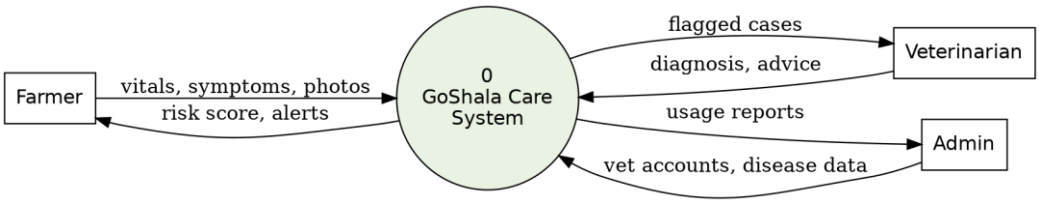


Figure 5.5: Data flow diagram, level 0 (context diagram)

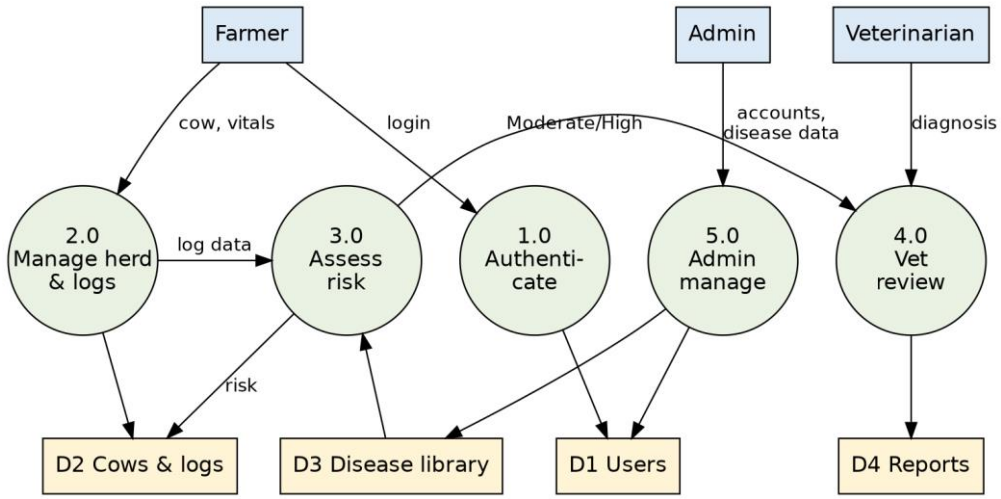


Figure 5.6: Data flow diagram, level 1

5.6 Sequence Diagram

Figure 5.6 illustrates the sequence of interactions involved in the cattle-health risk assessment and veterinarian review process. The farmer enters the cow's health parameters and symptoms through the web interface. The Flask backend receives the submitted information and sends it to the risk-assessment module, where rule-based analysis and machine-learning prediction are performed. The resulting risk level is stored in the MySQL database. If the cow is classified as Moderate or High Risk, the case is added to the veterinarian's review queue. The veterinarian reviews the cow's health information and records observations or recommendations, which are stored in the database and made available to the farmer through the application.

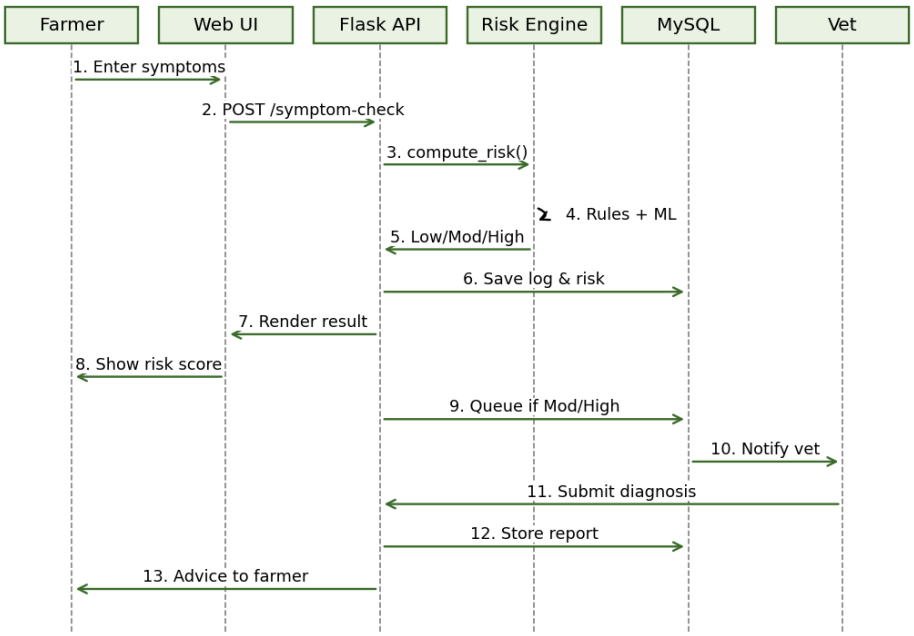


Figure 5.7: Sequence diagram: symptom check, risk score and vet review

CHAPTER 6: IMPLEMENTATION

6.1 Module Overview

The application is divided into modules that correspond to the functional requirements. Table 6.1 summarises the modules.

Table 6.1: Modules of GoShala Care

Module	Description
Authentication and RBAC	Registration, login, session handling and role-restricted routes.
Herd and vitals management	Adding cows and recording daily vitals and milk yield.
Symptom checker	Symptom checklist evaluated by the hybrid risk engine.
Photo scan	Image upload and CNN-based skin/udder screening.
Disease library	Reference information on diseases, symptoms and treatment.
Vet directory and map	Veterinarian listings displayed on a map.
Admin panel	Management of vets, disease library and directory.

6.2 Authentication and Role-Based Access Control

Users register and log in with an email and password. Passwords are stored as hashes. After login, the user id and role are stored in the session. A decorator restricts each route to the roles that may access it, and unauthorised access returns a 403 error.

Listing 6.1: Role-based access decorator (representative)

```
from functools import wraps
from flask import session, redirect, url_for, abort


def role required(*roles):
    def wrapper(fn):
        @wraps(fn)
        def inner(*args, **kwargs):
            if "user id" not in session:
                return redirect(url for("login"))
            if session.get("role") not in roles:
                abort(403)
            return fn(*args, **kwargs)
        return inner
    return wrapper
```

6.3 Herd and Vitals Management

Farmers register each cow with its tag number, breed and age. Each day the farmer records body temperature, feed intake, water intake and milk yield. Values are validated on the server for range and type before being stored. The dashboard displays the latest values and trends for each cow.

6.4 Symptom Checker and Risk Scoring

The symptom checker presents a checklist and combines the ticked symptoms with the latest vitals. The risk engine first applies rule-based thresholds and weights, then obtains a prediction from the machine learning model, and returns the higher of the

two levels. This keeps the result explainable, because the rule reasons can be shown to the farmer, while benefiting from the learned patterns in the ML model.

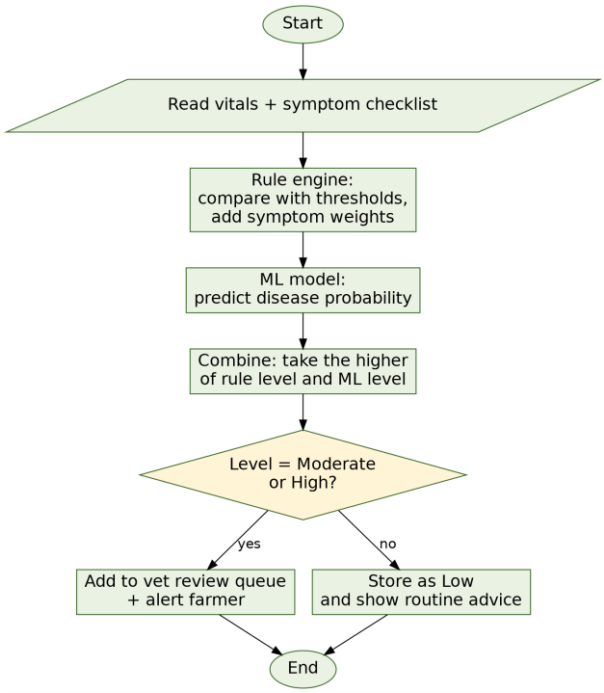


Figure 6.1: Flowchart of the hybrid risk scoring process

Listing 6.2: Risk-scoring function (representative; replace with your actual code)

```
LEVELS = ["Low", "Moderate", "High"]
NORMAL_TEMP = (38.0, 39.3)   # deg C, adult cattle (adjust as needed)

def compute_risk(log, symptoms, ml_level=None):
    score, reasons = 0, []
    t = log["temperature"]
    if t >= 40.0:
        score += 3; reasons.append("High fever")
    elif t > NORMAL_TEMP[1]:
        score += 2; reasons.append("Raised temperature")
    if log["feed change pct"] <= -30:
        score += 2; reasons.append("Large drop in feed intake")
    if log["milk change pct"] <= -20:
        score += 2; reasons.append("Milk yield drop")
    for s in symptoms:
        score += SYMPTOM_WEIGHTS.get(s, 1)
    rule = "High" if score >= 7 else "Moderate" if score >= 3 else "Low"
    final = rule
    if ml_level and LEVELS.index(ml_level) > LEVELS.index(rule):
        final = ml_level
    return final, score, reasons
```

6.5 Photo Scan Module

The photo scan module accepts an uploaded image of the skin or udder, resizes it to the input size of the model, and passes it to the CNN. The predicted class and confidence are shown to the farmer, and low-confidence or disease-positive results are added to the veterinarian queue. The models are loaded once when the application starts.

Listing 6.3: Model loading and prediction (representative)

```
import joblib, numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image

tab_model = joblib.load("models/tabular_classifier.pkl")
```

```
img_model = load_model("models/skin_udder_cnn.h5")
CLASSES = ["<class 1>", "<class 2>", "<class 3>"]   # your labels


def predict photo(path, size=(224, 224)):
    img = image.load img(path, target size=size)
    x = np.expand dims(image.img to array(img) / 255.0, 0)
    probs = img_model.predict(x)[0]
    k = int(np.argmax(probs))
    return CLASSES[k], float(probs[k])
```

6.6 Disease Library, Vet Directory and Admin Panel

The disease library stores each disease with its description, symptoms and recommended actions, and is searchable by farmers. The vet directory lists veterinarians with clinic details and coordinates and displays them as markers on a map so that farmers can find the nearest one. The Admin panel allows the Admin to create and deactivate vet accounts, edit the vet directory and maintain the disease and symptom library.

6.7 Veterinarian Review Workflow

When a log or photo scan yields a Moderate or High result, a report record is created with status Pending. The veterinarian sees a queue of pending cases with the cow details, recent vitals, symptoms and risk reasons. After reviewing, the vet enters a diagnosis and advice, the status changes to Reviewed, and the farmer sees the advice on the cow's page.

CHAPTER 7: DATA SCIENCE METHODOLOGY

7.1 Overview

Two models were developed: a tabular classifier that uses vitals and symptoms, and an image classifier that uses transfer learning. Figure 7.1 shows the overall pipeline from data to deployment.

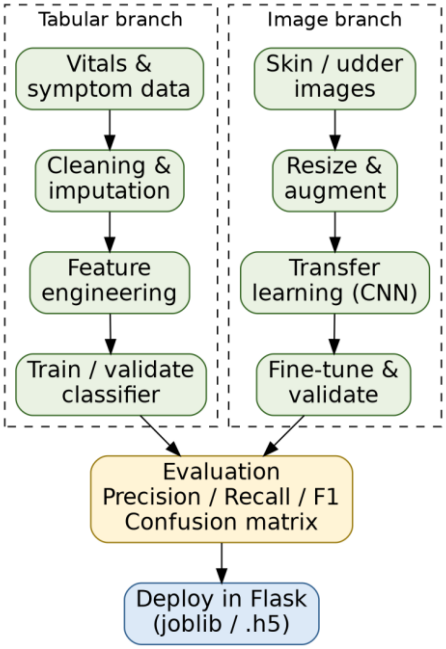


Figure 7.1: Machine learning pipeline for tabular and image models

7.2 Data Cleaning

The tabular data were checked for missing values, duplicate rows and impossible values such as temperatures outside plausible biological limits. Missing numeric values were imputed and categorical symptom fields were encoded as binary indicators. Images were checked for corrupt files, resized to a fixed size and normalised.

7.3 Feature Engineering

In addition to raw daily values, trend features were computed for each cow, including rolling averages of temperature, feed intake, water intake and milk yield over recent days, and the percentage change relative to the cow's own baseline. A cow-specific baseline is useful because normal values vary between animals. A symptom count and weighted symptom score were also included. Image data were augmented with rotation, flipping and brightness changes to increase variety.

7.4 Model Selection and Training

For the tabular health-risk prediction, a **Random Forest classifier** was trained using cattle health parameters and symptom-related features. The model was trained on the prepared dataset and evaluated using accuracy, precision, recall and F1-score. For the image-based component, a **transfer-learning approach using a pretrained CNN model** was used to classify the selected cattle-image categories. The trained models were saved and integrated with the Flask application to generate predictions for newly entered cattle-health data and uploaded images.

7.5 Evaluation Metrics

Precision, recall and F1-score are used for evaluation, with a confusion matrix to show the types of error. Precision is the fraction of predicted positive cases that are truly positive, recall is the fraction of actual positive cases that are found, and F1 is their harmonic mean. Recall is especially important in this application because a missed sick animal is more costly than a false alarm.

7.6 Interpretation of Results

The model performance was evaluated using accuracy, precision, recall and F1-score on the test data. The results indicate how effectively the models identify the different health-risk or disease classes. The tabular model uses cattle health parameters and symptoms, while the image model analyses visual features when applicable. The rule-based layer provides an additional assessment based on predefined health conditions, while the machine-learning layer identifies patterns across multiple input features. Together, these components support a more comprehensive health-risk assessment and can assist in identifying cattle that may require veterinary attention.

CHAPTER 8: TESTING

8.1 Testing Strategy

Testing was carried out at the unit and functional levels. Unit tests checked individual functions such as the risk scoring function and input validation. Functional tests exercised complete user flows for each role through the web interface. Test results should be recorded in the Actual Result and Status columns after execution.

8.2 Test Cases

Table 8.1: Test cases

ID	Test case	Input / action	Expected result	Status
TC1	Valid login	Correct email and password	User reaches role dashboard	[Pass/Fail]
TC2	Invalid login	Wrong password	Error shown; no session created	[Pass/Fail]
TC3	Role access control	Farmer opens admin URL	Access denied (403)	[Pass/Fail]
TC4	Role access control	Vet opens farmer log page	Access denied (403)	[Pass/Fail]
TC5	Add cow	Valid tag, breed, age	Cow saved and listed	[Pass/Fail]
TC6	Add cow validation	Missing tag number	Validation error shown	[Pass/Fail]
TC7	Log vitals	Valid daily values	Log stored and shown on dashboard	[Pass/Fail]
TC8	Vitals validation	Temperature out of valid range	Entry rejected with message	[Pass/Fail]
TC9	Trigger alert	High temperature with symptoms	Risk = High; alert generated	[Pass/Fail]
TC10	Symptom checker	Select several symptoms	Risk level and reasons displayed	[Pass/Fail]
TC11	Photo scan	Upload valid image	Class and confidence displayed	[Pass/Fail]
TC12	Photo scan invalid file	Upload non-image file	File rejected with message	[Pass/Fail]
TC13	Vet queue	Moderate/High case created	Case appears in vet queue	[Pass/Fail]
TC14	Vet review	Vet submits diagnosis	Status Reviewed; farmer sees advice	[Pass/Fail]
TC15	Admin vet management	Admin adds a vet	Vet appears in directory and map	[Pass/Fail]

8.3 Observations

The system was tested using functional test cases covering login, role-based access, cow management, health-data entry, risk assessment, photo scanning and veterinarian review. Any issues identified during testing were corrected through input validation, access-control checks and error handling. The final testing results are recorded in Table 8.1 based on the actual execution of the system.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Application Screens

This section presents the main screens of the implemented system.

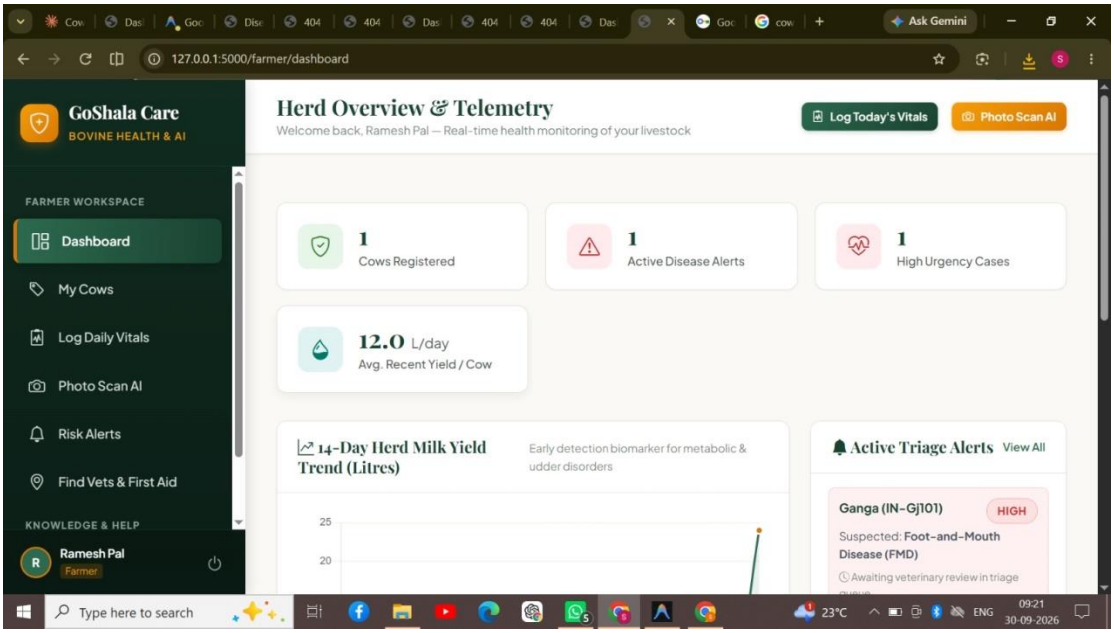


Figure 9.1: DashBoard Page

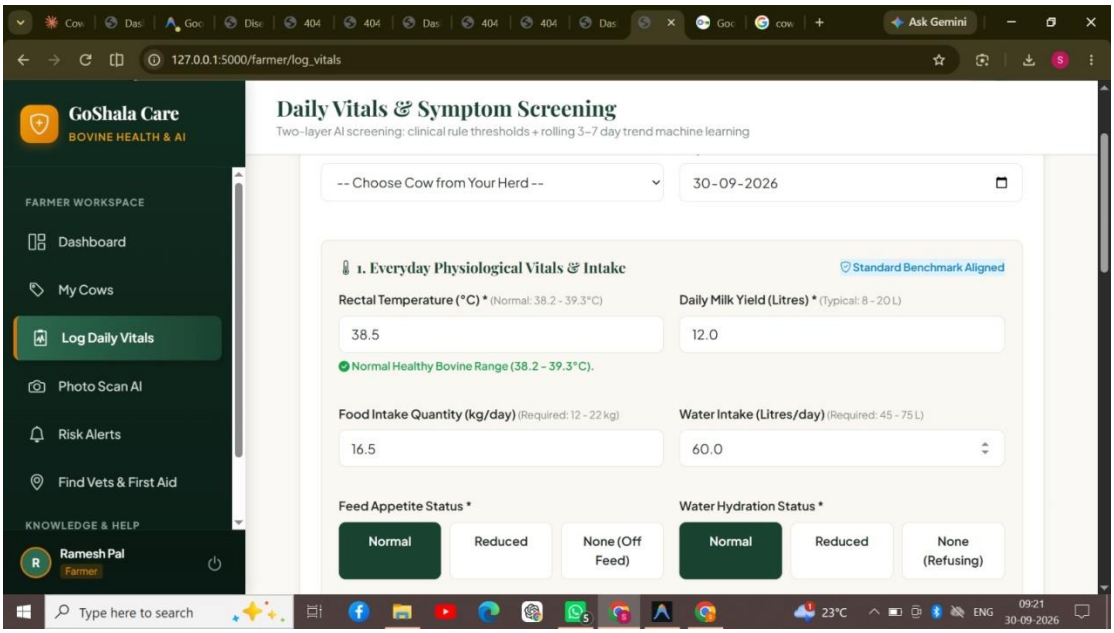


Figure 9.2: LogDaily Vitals Page

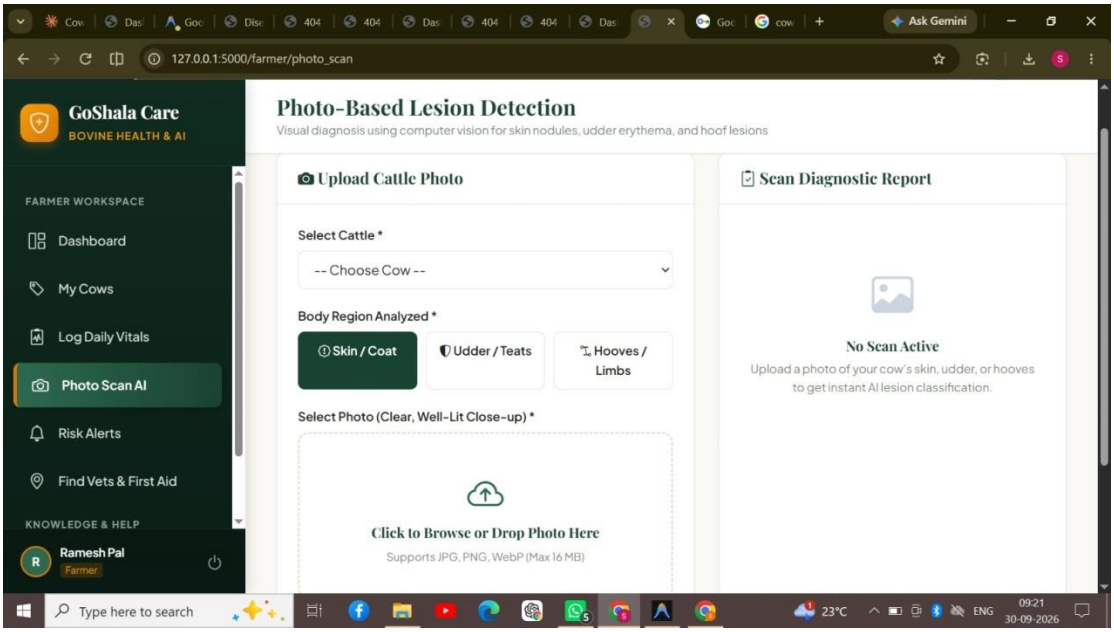


Figure 9.3: Photo Scan Page

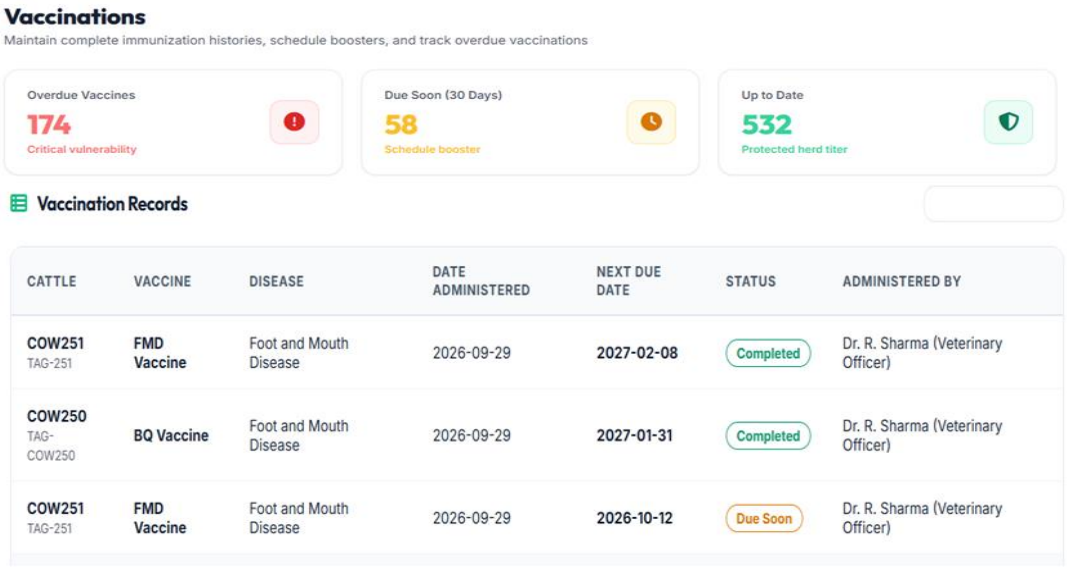


Figure 9.4: Vaccination Detail Page

9.2 Discussion

GoShala Care demonstrates the feasibility of providing early cattle-health screening through a web-based system without requiring dedicated IoT hardware. The combination of rule-based assessment and machine-learning prediction allows health information to be analysed using multiple indicators. The role-based design also provides separate functions for farmers, veterinarians and administrators, while the

veterinarian review workflow allows higher-risk cases to be examined by a professional. The testing process showed that the major modules could be integrated into a single application for cattle-health monitoring and management.

9.3 Limitations

- Health information is entered manually, so the quality of assessment depends on the accuracy and regularity of the farmer's entries.
- Model performance depends on the size, quality and representativeness of the datasets used for training.
- The image-based screening module is limited to the disease or condition classes included in the training dataset.
- Image predictions can be affected by factors such as lighting, image quality, camera angle and background.
- The system provides **screening and risk assessment and does not replace professional veterinary diagnosis**.
- The system currently has limited multilingual support and can be extended with regional-language interfaces in future.
- The system does not currently use real-time IoT sensor data; continuous sensor-based monitoring can be considered as a future enhancement.

CHAPTER 10: CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

GoShala Care is a complete, role-based web application for early detection and management of cow diseases that does not depend on IoT hardware. It allows farmers to record daily vitals and symptoms, combines rule-based thresholds with trained machine learning models to produce a Low, Moderate or High risk score, supports photo-based screening, and routes serious cases to veterinarians for review. The Admin module manages veterinarians and reference knowledge. The project shows that an affordable, explainable and practical system can help farmers detect problems earlier and connect with veterinary care.

10.2 Future Scope

- IoT sensor integration: optional collars or temperature sensors to automate data entry.
- Outbreak mapping: aggregating alerts by location to identify and warn about local outbreaks.
- Multilingual support: Kannada, Hindi and other regional languages for wider adoption.
- Chatbot assistant: a conversational assistant to guide farmers through symptom entry and advice.
- Mobile application: a mobile app with offline data entry and notifications.
- Larger datasets and additional diseases to improve model coverage and accuracy.

REFERENCES

[1] M. Alipio and M. L. Villena, "Intelligent wearable devices and biosensors for monitoring cattle health conditions: A review and classification," *Smart Health*, vol. 27, Art. no. 100369, 2023, doi: 10.1016/j.smhl.2022.100369.

[2] Y. Wang, Q. Li, M. Chu, X. Kang, and others, "Application of infrared thermography and machine learning techniques in cattle health assessments: A review," *Biosystems Engineering*, vol. 230, pp. 361–387, 2023, doi: 10.1016/j.biosystemseng.2023.05.002.

[3] N. Slob, C. Catal, and A. Kassahun, "Application of machine learning to improve dairy farm management: A systematic literature review," *Preventive Veterinary Medicine*, vol. 187, Art. no. 105237, 2021, doi: 10.1016/j.prevetmed.2020.105237.

[4] "Early Detection of Lumpy Skin Disease in Cattle Using Deep Learning—A Comparative Analysis of Pretrained Models," 2024.

[5] R. C. M. Santana, E. da Silva Guimarães, F. D. Caracuschanski, L. C. Brassolatti, M. L. da Silva, A. R. Garcia, J. R. M. Pezzopane, T. C. Alves, P. Tholon, M. V. dos Santos, and L. F. Zafalon, "Machine Learning Techniques Associated With Infrared Thermography to Optimize the Diagnosis of Bovine Subclinical Mastitis," *Veterinary Medicine International*, vol. 2025, Art. no. 5585458, 2025, doi: 10.1155/vmi/5585458.

[6] P. K. Astuti, Z. Bagi, E. Mikó, V. Bilicki, G. Sudár, and S. Kusza, "Dairy farming sustainability: A systematic review to understand mastitis and lameness detection using machine learning (ML)," *Annals of Agricultural Sciences*, vol. 71, no. 1, Art. no. 100412, 2026, doi: 10.1016/j.aoas.2026.100412.

APPENDIX

A. Database Schema

The following is a representative schema

```
CREATE TABLE users (user id INT AUTO INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
  email VARCHAR(120) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL,
  role ENUM('farmer','vet','admin') NOT NULL);
CREATE TABLE cows (cow id INT AUTO INCREMENT PRIMARY KEY, owner id INT NOT NULL,
  tag_no VARCHAR(30), breed VARCHAR(50), age INT,
  FOREIGN KEY (owner_id) REFERENCES users(user_id));
CREATE TABLE vitals_symptom_logs (log_id INT AUTO_INCREMENT PRIMARY KEY,
    cow id INT NOT NULL,
  temperature DECIMAL(4,1), feed intake DECIMAL(6,2), water intake DECIMAL(6,2),
  symptoms TEXT, risk_level ENUM('Low','Moderate','High'), log_date DATE,
  FOREIGN KEY (cow_id) REFERENCES cows(cow_id));
CREATE TABLE milk logs (milk id INT AUTO INCREMENT PRIMARY KEY, cow id INT NOT NULL,
  yield litres DECIMAL(5,2), log date DATE,
      FOREIGN KEY (cow id) REFERENCES cows(cow id));
CREATE TABLE diseases (disease_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100),
  description TEXT, treatment TEXT);
CREATE TABLE symptoms (symptom id INT AUTO INCREMENT PRIMARY KEY,
    name VARCHAR(100));
CREATE TABLE disease_symptoms (disease_id INT, symptom_id INT,
  FOREIGN KEY (disease id) REFERENCES diseases(disease id),
  FOREIGN KEY (symptom id) REFERENCES symptoms(symptom id));
CREATE TABLE vet directory (vet id INT AUTO INCREMENT PRIMARY KEY, user id INT,
  clinic VARCHAR(120), latitude DECIMAL(9,6), longitude DECIMAL(9,6),
  FOREIGN KEY (user id) REFERENCES users(user id));
CREATE TABLE reports (report id INT AUTO INCREMENT PRIMARY KEY, log id INT,
    vet id INT,
  diagnosis TEXT, status ENUM('Pending','Reviewed') DEFAULT 'Pending',
  FOREIGN KEY (log id) REFERENCES vitals symptom logs(log id),
  FOREIGN KEY (vet_id) REFERENCES vet_directory(vet_id));
```